Exercises for the Lecture Advanced Methods of AI

Asking, Defining, Constraining, Deriving, Generating

Tutorial: Thursday, 12.11.2026, 8:30–10:00, room 0.30, ECO 5

Last week you *represented* the pizza world in first-order logic and in Python. Tonight a service robot in a pizzeria *uses* that knowledge. Everything it does starts with one kind of expression, a **description**, written in the Entity Query Language (EQL). The five acts of the evening are five things the robot does with one:

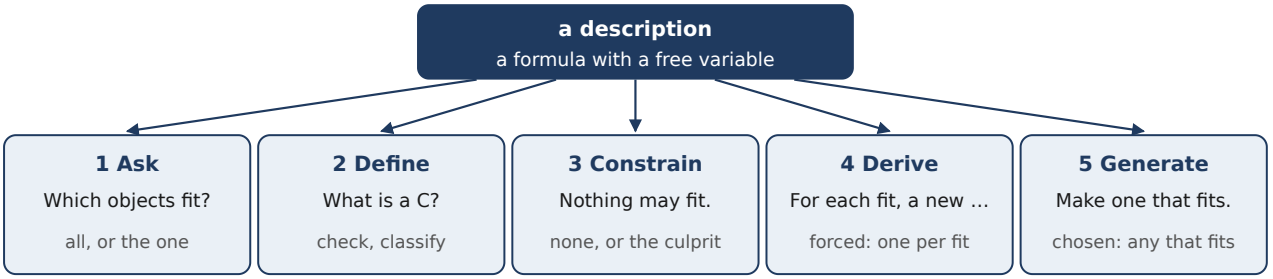


Figure 1. One description, five uses. The expression stays the same; what changes is what is done with it. Asking and constraining *read* what the robot knows; defining, deriving and generating *write* to it.

How to work

The evening happens in the notebook `03-pizza_queries.ipynb` (link on Stud.IP). Open it in pairs, run its setup cell, and work through it from top to bottom.

Three times, the tutor calls a **pen break**. Put the laptop aside and turn to this sheet: write the logic by hand for five minutes. Then the tutor writes an answer on the board and takes your questions, and you go back to the notebook. Questions marked ★ are for pairs who finish early.

PEN BREAK 1 · AFTER ACT 1

Asking

5 minutes of writing, then the board.

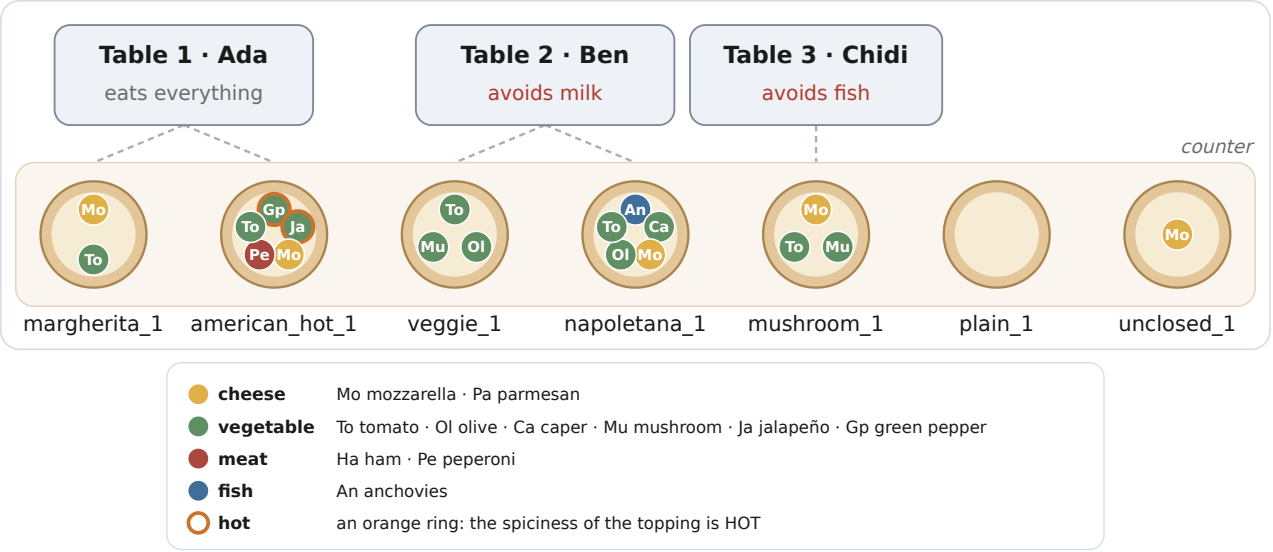


Figure 2. What the robot knows at 19:30. Lines connect each table to the pizzas it ordered.

(a) The picture is an interpretation

Figure 2 is an *interpretation* of first-order logic. What is its domain? What is the extension of the predicate CheeseTopping, and that of the relation hasTopping?

(b) Some, only, none

```
Q1 = a(Pizza)(toppings=every_is(VegetableTopping)).from_(pizzas)
Q2 = a(Pizza)(toppings=[a(MozzarellaTopping), a(TomatoTopping)]).from_(pizzas)
```

Write Q1 as a formula with the free variable *x*. The robot returned plain_1, a pizza without any topping: show with your formula why plain_1 satisfies it. Then write Q2 as a formula.

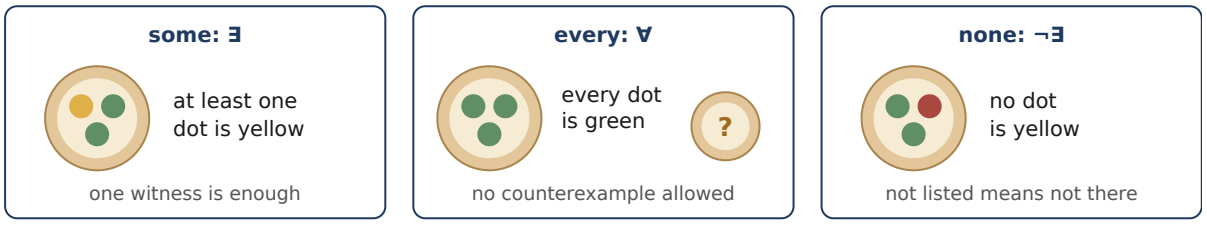


Figure 4. The three quantifier patterns, as the robot checks them on one pizza. The small pizza has no toppings at all: is every dot on it green?

(c) Following the relations

Write `hot_orders`, the orders that contain a pizza with a hot topping, as a formula with the free variable *o*. The query `set_of(order.guest, pizza, topping)` of the notebook had two answers where `hot_orders` had one: which variables are free in each?

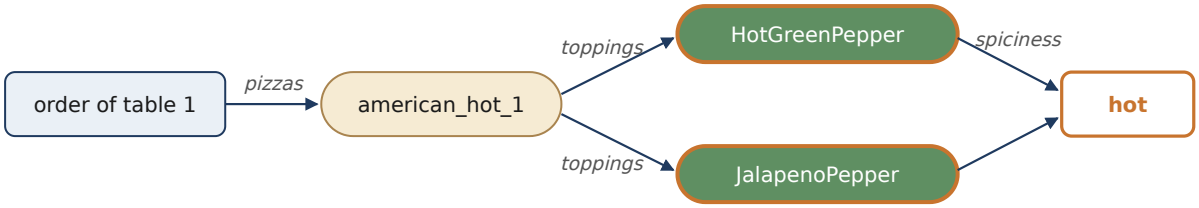


Figure 5. The paths from Ada's order to a hot topping. There are two.

(d) Making a description precise

"The pizza with olives" fitted two pizzas; Ben wants the one *without fish*. Write the precise description as a formula.

> ★ **THINK**
>
> Last week the OWL reasoner refused to classify UnclosedPizza (a pizza with some mozzarella) as vegetarian. Tonight the query `vegetarian` returned unclosed_1. Who is right, the query or the reasoner?

PEN BREAK 2 · AFTER ACT 3

Defining and constraining

5 minutes of writing, then the board.

(a) A class declaration says part of what a thing is

Cell 2(a) of the notebook printed what KRROOD reads from the class declarations. Write each line as a formula. Can any of them ever make an object a Pizza, or a VegetarianPizza?

Read from the declarations	First-order formula
every MozzarellaTopping is a CheeseTopping	$\forall x$ (MozzarellaTopping(x) $\rightarrow$ CheeseTopping(x))
every Pizza has toppings, each a PizzaTopping	
every Order has a guest, which is a Guest	
every MenuLabel is a role of one Pizza	

Hint: "has *r*, each a B" is a *relation* $r(x, y)$ between an A and each of its Bs; "has a *r*, which is a B" is a *function* $r(x)$ whose value is a B; "is a role of one B" is a function from the role to the B that takes it. For the question, look at what stands on the right of $\rightarrow$.

(b) Concluding from a name

The menu's definition from the notebook: $\forall x$ (VegetarianPizza(x) $\leftrightarrow$ Pizza(x) $\wedge$ $\neg\exists t$ (hasTopping(x, t) $\wedge$ (MeatTopping(t) $\vee$ FishTopping(t)))). A pizza from the other branch arrives with a green V; nobody wrote down its toppings. What may the robot tell Chidi, who avoids fish? And Ben, who avoids milk? Which direction of $\leftrightarrow$ do you use?

(c) A constraint is a query that must have no answer

The kitchen's rule: every pizza of an order is safe for the guest of that order. Write it as a formula (with hasPizza, guest and IsSafeFor), and then its negation, with the $\neg$ pushed inwards. Which of the two did `dangerous` ask?

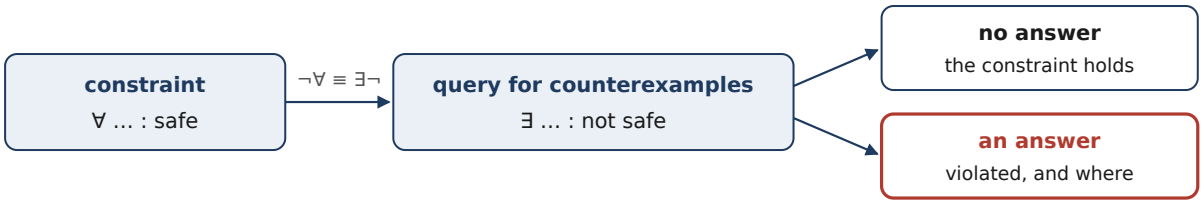


Figure 6. A universal statement is violated exactly when a counterexample exists.

> **★ THINK**
>
> Suppose the kitchen's rule were an axiom of last week's ontology. What would the reasoner report for tonight's facts? And if the mozzarella of the Napoletana had simply not been recorded?

PEN BREAK 3 · AFTER ACT 5

Deriving, generating, and the whole evening

5 minutes of writing for (a) and (b); the summary together with the tutor.

(a) A rule with ∃ in its head

The milk rule of Act 4 is

```
∀x (Pizza(x) ∧ ∃t (hasTopping(x, t) ∧ CheeseTopping(t)) → ∃w (Warning(w) ∧ about(w, x) ∧ allergen(w) = milk))
```

Write the fish rule the same way. The label rule of Act 2, $\forall x\,(\varphi(x) \rightarrow \text{VegetarianPizza}(x))$, only labelled pizzas that were there; the milk rule created five new warnings. What in the formula makes the difference?

(b) One plan, many answerers

The robot's next task is a plan with open slots, each written as a description:

Bring ① a pizza that is safe for Ben, lying ② at some pose on the counter, **to** ③ the table of Ben, **holding the plate with** ④ a grasp that works. **If none is on the counter:** ⑤ have one made.

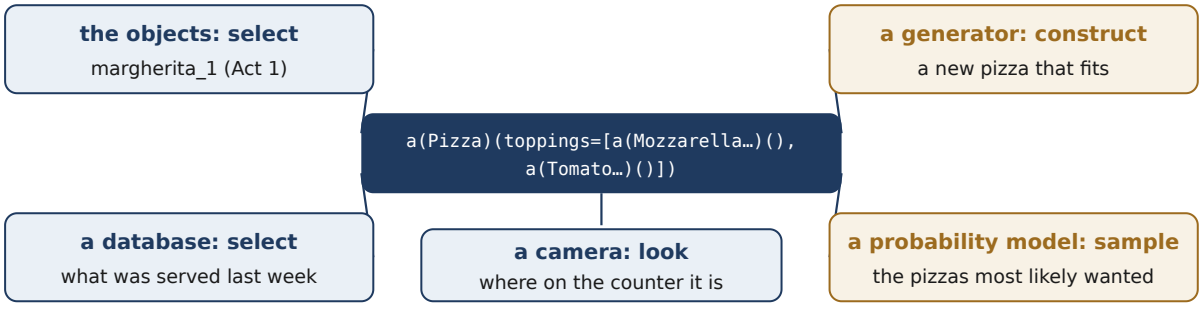


Figure 10. The description is written once, symbolically. Selecting (blue) and constructing or sampling (ochre) are different ways of answering it.

Which kind of backend (Figure 10) should answer each slot? Choose from **K** a query over the knowledge base, **R** rules and predicates, **L** looking with the camera, **S** sampling from a probability model, **G** a generator.

① safe pizza	② its pose	③ Ben's table	④ grasp	⑤ have one made

(c) The whole evening

For each use of a description, give tonight's example and what it is in logic.

Use	The question behind it	Tonight's example	In logic
1 Ask	Which objects fit?		
2 Define	What is a C?		
3 Constrain	Does anything violate it?		
4 Derive	What new object does each fit get?		
5 Generate	What could fit?		

Take home

One expression, a description, carried the whole evening. Its meaning is a formula with free variables; what you *do* with it decides whether it is a question, a definition, a constraint, the body of a rule or a specification, and which backend answers it decides whether it is checked, computed or constructed.

To think about at home: a robot tidies a kitchen. Give one description for each of the five uses, and the backend that should answer it.